

TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN A DISTANCIA
Universidad Tecnológica Nacional (UTN)

Trabajo Práctico Integrador
Programación 2

Food Store

Sistema de Gestión de Pedidos de Comida

Alumno: Medina, Ivan Andres

Materia: Programación 2

Carrera: Tecnicatura Universitaria en Programación

Institución: Universidad Tecnológica Nacional (UTN)

Fecha: 22 de junio de 2026

Índice

1. Marco Teórico

- 1.1. Lenguaje y entorno
- 1.2. Programación Orientada a Objetos en Java
- 1.3. Colecciones
- 1.4. Manejo de Excepciones

2. Modelo del Sistema (UML y Entidades)

3. Decisiones Técnicas y Arquitectura

- 3.1. Estructura de paquetes
- 3.2. Herencia y clase Base
- 3.3. Relaciones bidireccionales entre entidades
- 3.4. Interfaz Calculable
- 3.5. Soft Delete (Baja Lógica)
- 3.6. Excepciones propias

4. Funcionalidades Opcionales y Mejoras de UX

- 4.1. Funcionalidades opcionales de la consigna implementadas
- 4.2. Listado previo en operaciones de edición, eliminación y filtrado
- 4.3. Validaciones robustas con repregunta (do-while)
- 4.4. Opción 5 – Volver al menú principal
- 4.5. Mensajes claros de error y éxito

5. Dificultades y Soluciones

- 5.1. Relaciones bidireccionales y consistencia
- 5.2. Integridad al eliminar un Producto
- 5.3. Manejo de stock durante la creación de un pedido
- 5.4. Decisión sobre la estrategia de generación de IDs

6. Links del Proyecto

7. Bibliografía / Webgrafía

1. Marco Teórico

1.1. Lenguaje y entorno

El sistema fue desarrollado en **Java 21**, un lenguaje orientado a objetos multiplataforma que corre sobre la Java Virtual Machine (JVM). Java ofrece tipado estático, recolección de basura automática y una biblioteca estándar muy completa. El entorno de desarrollo utilizado fue **NetBeans IDE**, que facilita la organización del código por paquetes y la ejecución del programa desde consola.

1.2. Programación Orientada a Objetos (POO)

Los cuatro pilares de la POO aplicados en este proyecto son:

- **Encapsulamiento:** atributos privados accesibles mediante getters y setters con validaciones incorporadas.
- **Herencia:** la clase abstracta **Base** concentra los atributos comunes (id, eliminado, createdAt) y todas las entidades la extienden.
- **Polimorfismo:** el método abstracto **toString()** es sobrescrito en cada entidad para producir representaciones útiles en consola.
- **Abstracción:** la interfaz **Calculable** define el contrato de **calcularTotal()**, implementado por la clase **Pedido**.

1.3. Colecciones

Este proyecto utiliza **ArrayList** para almacenar las entidades durante la ejecución. Los getters exponen las listas como **Collections.unmodifiableList()** para proteger la integridad interna de cada entidad, forzando el uso de los métodos definidos para agregar o eliminar elementos.

1.4. Manejo de Excepciones

Se optó por extender **RuntimeException** para todas las excepciones propias, evitando la obligación de declararlas en cada firma de método y simplificando el flujo de control en los menús. Cada excepción tiene un nombre semántico que describe el error de negocio que representa.

2. Modelo del Sistema (UML y Entidades)

El diseño sigue el modelo UML provisto por la cátedra. A continuación se describen las entidades principales y sus relaciones:

Base (abstracta)	Clase raíz. Atributos: id (Long, auto-incremental), eliminado (boolean), createdAt (LocalDateTime). Define toString() abstracto.
Categoria	Hereda de Base. Atributos: nombre, descripcion. Relación 1:N con Producto.
Producto	Hereda de Base. Atributos: nombre, precio, descripcion, stock, imagen, disponible. Relación N:1 con Categoria.
Usuario	Hereda de Base. Atributos: nombre, apellido, mail (único), celular, contrasenia, rol (Enum). Relación 1:N con Pedido.
Pedido	Hereda de Base. Implementa Calculable. Atributos: fecha, estado (Enum), total, formaPago (Enum). Composición 1:N con DetallePedido. Relación N:1 con Usuario.
DetallePedido	Hereda de Base. Atributos: cantidad, subtotal (calculado). Relación N:1 con Producto.

Enumeraciones utilizadas:

Rol	ADMIN, USUARIO
Estado	PENDIENTE, CONFIRMADO, TERMINADO, CANCELADO
FormaPago	TARJETA, TRANSFERENCIA, EFECTIVO

3. Decisiones Técnicas y Arquitectura

3.1. Estructura de paquetes

El proyecto se organiza en paquetes con responsabilidades bien definidas:

```
src/
  ■■■■ entities/      → Clases de dominio (Base, Categoria, Producto, Usuario, Pedido,
DetallePedido)
  ■■■■ enums/        → Enumeraciones (Rol, Estado, FormaPago)
  ■■■■ interfaces/    → Contratos (Calculable)
  ■■■■ exception/     → Excepciones propias
  ■■■■ transacciones/ → Lógica de negocio (Transaccion*)
  ■■■■ menus/         → Interacción con el usuario (SubMenu*, MenuPrincipal)
  ■■■■ validaciones/  → Validaciones de input (ValidacionInput)
  ■■■■ Main.java      → Punto de entrada
```

El **Main.java** solo instancia el menú principal; los **SubMenus** gestionan la interacción con el usuario; las clases **Transaccion*** concentran las reglas de negocio.

3.2. Herencia y clase Base

La clase abstracta **Base** centraliza un contador estático `numeradorId` que auto-incrementa el identificador de cada entidad. Todas las entidades heredan de ella, evitando duplicación de código para los atributos comunes (`id`, `eliminado`, `createdAt`).

3.3. Relaciones bidireccionales entre entidades

Las relaciones **Categoria** ↔ **Producto** y **Usuario** ↔ **Pedido** son bidireccionales y se mantienen consistentes mediante métodos dedicados. Cada método verifica el estado actual de la relación antes de modificarla, previniendo referencias circulares y duplicados.

3.4. Interfaz Calculable

La interfaz **Calculable** define el método `calcularTotal()`, implementado por **Pedido**. Se recalcula automáticamente cada vez que se agrega o elimina un detalle. El uso de interfaz permite que otras entidades adopten el mismo contrato en el futuro.

3.5. Soft Delete (Baja Lógica)

Ninguna entidad se remueve físicamente. Se marca `eliminado = true` (de **Base**). Los métodos de listado filtran los eliminados y los de búsqueda retornan null para ellos, preservando el historial de datos.

3.6. Excepciones propias

Se implementaron 7 excepciones personalizadas extendiendo **RuntimeException**:

EntidadNoEncontradaException	Búsqueda por ID que no existe o está eliminada.
EliminacionEntidadException	Intento de eliminar una entidad con dependencias activas.
NombreCategoriaRepetidoException	Nombre de categoría ya existente en el sistema.
MailDeUsuarioRepetidoException	Mail de usuario ya registrado.

StockInvalidoException	Stock negativo o insuficiente al agregar un detalle.
PrecioInvalidoException	Precio negativo al crear o editar un producto.
CantidadDetallePedidoException	Cantidad menor o igual a cero en un DetallePedido.

4. Funcionalidades Opcionales y Mejoras de UX

Más allá de los requisitos obligatorios, se implementaron funcionalidades opcionales de la consigna y decisiones propias orientadas a mejorar la experiencia del usuario.

4.1. Funcionalidades opcionales de la consigna implementadas

HU-CAT-03 – Editar	El sistema lista todas las categorías antes de solicitar el ID a editar.
HU-PROD-01 – Listar	Dos modos: listado general o filtrado por nombre de categoría, a elección del usuario.
HU-USR-02 – Crear usuario	Validación de formato básico de mail: se verifica que contenga el carácter '@'.
HU-PED-01 – Listar pedidos	Filtrado por usuario, mostrando previamente la lista de usuarios disponibles.

4.2. Listado previo en operaciones de edición, eliminación y filtrado

Siempre que el usuario necesita ingresar un ID para editar, eliminar o filtrar, el sistema primero muestra el listado completo de entidades disponibles. Esto aplica en todos los submenús, evitando que el usuario deba recordar IDs de memoria y reduciendo errores de entrada.

4.3. Validaciones robustas con repregunta (do-while)

Todas las entradas están protegidas por estructuras do-while que no avanzan hasta recibir un input válido, centralizadas en **ValidacionInput**. Se cubren:

- **Strings no vacíos y no puramente numéricos** — para nombres y descripciones.
- **Celular solo numérico** — validado con `matches("\\d+")`.
- **Mail con '@'** — validación de formato básico.
- **Long/Double/Int con rangos** — para IDs, precios y stock, con captura de `NumberFormatException`.
- **Opciones de menú** — con mensajes diferenciados para número fuera de rango vs. carácter no numérico.

4.4. Opción 5 – Volver al menú principal

Todos los submenús incluyen una **opción 5 – Menú principal** explícita, usada como condición de corte del do-while de navegación. Garantiza que el usuario siempre tenga una salida clara desde cualquier submenú.

4.5. Mensajes claros de error y éxito

- **Éxito:** cada operación CRUD exitosa confirma con el ID del objeto afectado (ej: *"¡¡¡Categoría con id: #3 creada exitosamente!!!"*).
- **Error de negocio:** las excepciones muestran mensajes descriptivos (ej: *"Ya existe una categoría con el nombre: PIZZAS"*).
- **Sin cambios:** si no se registran modificaciones, se informa explícitamente al usuario.
- **Confirmación antes de eliminar:** todas las bajas lógicas requieren confirmar con 'S', previniendo eliminaciones accidentales.

5. Dificultades y Soluciones

5.1. Relaciones bidireccionales y consistencia

Al establecer relaciones bidireccionales surgía el riesgo de llamadas recursivas infinitas. La solución fue agregar una guarda de igualdad al inicio de cada setter (if (this.categoria != categoriaNueva)) que corta el ciclo antes de que se produzca.

5.2. Integridad al eliminar un Producto

Antes de hacer el soft delete, se verifica si el producto aparece en el detalle de algún pedido no eliminado. Si es así, se lanza EliminacionEntidadException, preservando la integridad de los pedidos existentes.

5.3. Manejo de stock durante la creación de un pedido

La consigna indica: "si ocurre una excepción al agregar un detalle, se debe capturar el error y cancelar la creación del pedido para no dejar datos inconsistentes". Se evaluó que cancelar automáticamente sería perjudicial para el usuario. Por eso, ante una excepción de StockInvalidoException o EntidadNoEncontradaException, el sistema **sí lanza y captura la excepción** (cumpliendo la consigna) pero ofrece tres opciones:

- **Opción 1 – Reintentar:** intentar agregar otro producto al pedido.
- **Opción 2 – Terminar:** conservar el pedido con los detalles ya cargados exitosamente.
- **Opción 3 – Cancelar:** eliminar el pedido completamente de la memoria.

La consistencia está garantizada en todas las casuísticas mediante dos métodos en **Producto**:

venderUnidades(int n)	Invocado al confirmar un detalle exitoso. Descuenta el stock. Si el resultado sería negativo, lanza StockInvalidoException sin modificar nada.
reponerStock(int n)	Invocado al cancelar un pedido (Opción 3). Devuelve al stock de cada producto las unidades descontadas, dejando el sistema en el estado previo.

Resultado: se cumple con el espíritu de la consigna (captura de excepción, sin datos inconsistentes) y se ofrece al usuario la flexibilidad de decidir cómo manejar el error.

5.4. Decisión sobre la estrategia de generación de IDs

Se evaluaron dos alternativas para la generación de IDs:

- **ID por clase:** cada clase tiene su propio contador. IDs consecutivos por clase, pero objetos de distintas clases pueden compartir el mismo valor numérico.
- **ID global en Base (elegida):** contador único compartido por todas las entidades. IDs únicos e irrepetibles en todo el sistema, mayor trazabilidad y coherencia. Como contrapartida, los IDs dentro de una misma clase pueden no ser consecutivos.

Decisión: se eligió el **ID global** por garantizar unicidad absoluta, simplificar la trazabilidad y preparar el modelo para una eventual migración a base de datos relacional. Como excepción documentada, **DetallePedido** usa su propio contador dado que sus instancias nunca se referencian por ID desde los menús.

6. Links del Proyecto

Repositorio GitHub	https://github.com/MedinalvanAndres/TPI-Programacion2
Video Demostración	https://drive.google.com/drive/folders/1h8ulhrO8VL6V85OL4UebMB616APp_zol

7. Bibliografía / Webgrafía

- [1] Oracle Corporation. (2024). *Java SE 21 Documentation*. <https://docs.oracle.com/en/java/javase/21/>
- [2] Oracle Corporation. (2024). *Java Collections Framework Overview*. <https://docs.oracle.com/javase/8/docs/technotes/guides/collections/overview.html>
- [3] Oracle Corporation. (2024). *Lesson: Exceptions – The Java Tutorials*. <https://docs.oracle.com/javase/tutorial/essential/exceptions/>
- [4] Bloch, J. (2018). *Effective Java* (3.a ed.). Addison-Wesley Professional.
- [5] Freeman, E., & Robson, E. (2020). *Head First Design Patterns* (2.a ed.). O'Reilly Media.
- [6] Cátedra de Programación 2, UTN. (2026). *Consigna y Pautas de Corrección del Trabajo Práctico Integrador – Food Store*.
- [7] Cátedra de Programación 2, UTN. (2026). *U8-A1-Complemento-Practico-Mas-alla-de-la-Herencia-Simple*.
- [8] Cátedra de Programación 2, UTN. (2026). *El-Arte-de-la-Adaptabilidad*.
- [9] Cátedra de Programación 2, UTN. (2026). *Diseno-de-Jerarquias-Complejas*.
- [10] Cátedra de Programación 2, UTN. (2026). *Control-y-Generalizacion*.
- [11] Cátedra de Programación 2, UTN. (2026). *El-Arte-de-No-Repetirse*.
- [12] Cátedra de Programación 2, UTN. (2026). *U8-A3-Complemento-Practico-Control-de-Danos-y-Robustez*.
- [13] Cátedra de Programación 2, UTN. (2026). *U8-A2-Complemento-Practico-Evolucion-de-las-Interfaces*.
- [14] Cimino, C. (2024). *OBJETOS (POO) desde -10 en Java* [Lista de reproducción]. YouTube. <https://www.youtube.com/playlist?list=PLOW7b-NX043aSC7ZNtEuVfY8xZoNzVqdJ>